

Cerebellum Inspired Adaptive Inverse Model with Spiking Neural Network

by

Bryan Fredrick Kusuma

<https://github.com/b-kus/cerebellum-adaptive-inverse-snn>

Declaration of AI Use

AI tools (including Claude, ChatGPT, and Deepseek) were used as coding assistants, for debugging, literature search and to aid in structuring and editing this report. All AI-generated content was critically reviewed and modified by the author.

Table of Content

Declaration of AI Use	1
Table of Content	2
Abstract	3
Introduction	3
Figure 1	4
Methodology	6
2.1 Overall Architecture	6
2.2 The Plant: 1-DoF Inverted Pendulum	7
2.3 The PID Controller	7
2.4 The Spiking Cerebellar Model (Brian2)	7
2.5 Learning and Training	8
2.6 Evaluation	8
2.7 Control Loop Architecture	9
Result	10
3.1 Experiment 1: PD ($K_i=0$) vs PD + SNN	10
Figure 1A	11
Table 1	11
3.2 Experiment 2: PID ($K_i=2.0$) vs PID + SNN	11
Figure 1B	12
Table 2	12
3.3 Visual Results	12
Figure 2A-D	13
3.4 Summary	13
Discussion	14
References	16

Abstract

The cerebellum is theorized to function as an internal model of the brain, particularly the inverse model, which plays a central role in motor learning and adaptation. This project implements a biologically plausible spiking neural network (SNN) model of the cerebellum, based on Kawato's Feedback-Error-Learning (FEL) theory, to control a simulated 2D pendulum with a single joint in MuJoCo. The network uses Gaussian population coding to encode the state into 9 mossy fibers, which project to 28 granule cells, and finally to 2 Purkinje cells arranged as an agonist-antagonist pair, where the difference in their firing rates decodes the torque command. Long-Term Depression (LTD) and homeostatic Long-Term Potentiation (LTP) are driven by the error signal from a PID controller. The SNN learns online, adapting its synaptic weights in real time to compensate for a 50 ms actuator transport delay that destabilises the PID alone. Our results show that with active learning, PD + SNN performs better than PD only, reducing the steady-state error decently. While it stays close to the target angle, the pendulum remains unstable and does not fully converge. On the other hand, PID only produces lower steady-state error than PID + SNN, suggesting that the SNN may help with PD only but may not be as useful when the integral term is already present. Further work is needed to improve the performance of the SNN.

Introduction

The cerebellum is one of the most structurally regular and evolutionarily conserved regions of the brain. Its remarkably uniform microcircuitry, composed of granule cells, Purkinje cells, and a well-defined set of afferent pathways, has made it an attractive target for computational modelling. While the cerebellum has long been associated with motor coordination and balance, a growing body of evidence suggests that its functional role extends to a broader set of computations, including motor learning, adaptation, and even cognitive processing (Ito, 2008).

A leading theoretical framework for understanding cerebellar function is the internal model theory, most prominently advanced by Wolpert and colleagues (Wolpert et al., 1998). According to this view, the brain constructs internal representations of the body and the external world to enable fast, accurate, and adaptive control. Two types of internal models are distinguished: forward models, which predict the sensory consequences of a motor command, and inverse models, which compute the motor command required to achieve a desired sensory state. The cerebellum is thought to be a primary candidate for implementing inverse models, particularly in the context of rapid, feedback-driven motor control.

The inverse model problem is deceptively simple: given a desired movement and the current state of the body, what motor command should be generated? In a biological system, this problem is complicated by two factors. First, the mapping from desired state to motor command is often non-linear, context-dependent, and ill-posed. Second, the brain does not have direct access to the "correct" motor command; it only has access to sensory errors—the difference between what was desired and what actually occurred. This raises a fundamental question: how can the cerebellum learn an inverse model when the teaching signal is not available in the correct format?

Figure 1

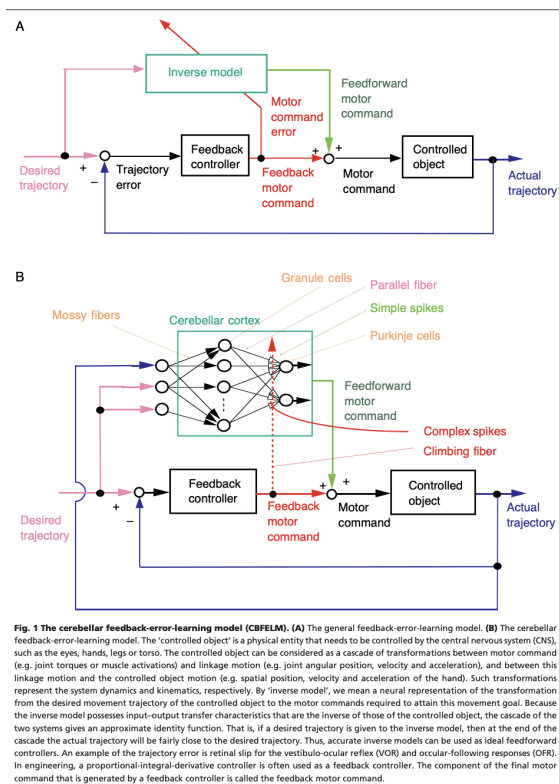


Figure 1: The model inspiration. Adapted from Wolpert, Miall, and Kawato (1998), Figure 1.

Kawato and colleagues proposed an elegant solution to this problem, known as Feedback-Error-Learning (FEL) (Kawato & Gomi, 1992). The key insight is that the output of a feedback controller—such as a proportional-integral-derivative (PID) controller—can serve as the teaching signal for the inverse model. In the cerebellar context, this feedback signal is thought to be conveyed by the climbing fibres, which originate in the inferior olive and synapse onto Purkinje cells. Thus, FEL provides a plausible mechanism

for transforming sensory errors into motor-command errors, thereby enabling supervised learning of the inverse dynamics.

The goal of this project is to implement a biologically plausible inverse model of the cerebellum using a spiking neural network (SNN). The main inspiration of the model is Kawato's Cerebellar Feedback Error Learning Model (CBFELM) (Wolpert et al., 1998) as can be seen from figure 1. While FEL has been successfully implemented in artificial neural networks (ANNs) (e.g., Gomi & Kawato, 1993), its implementation in spiking networks remains relatively unexplored. SNNs offer several advantages over ANNs: they operate on discrete events (spikes), capture the temporal dynamics of neural processing, and are more directly amenable to neuromorphic hardware. However, they also introduce significant challenges, including the need for temporally precise encoding of continuous signals, the development of stable plasticity rules, and the management of spike-induced noise in real-time control loops.

In this work, we address the following specific problems:

- Cerebellum circuit modeling: How to model such complex networks of different types of cell types and connection of cerebellum?
- Encoding and decoding: How can continuous state variables (e.g., angle, angular velocity, target angle) be reliably encoded into spike trains, and how can the resulting spikes be decoded into a continuous torque command?
- Learning in a spiking framework: Can we implement Kawato's FEL using biologically plausible plasticity rules (LTD and homeostatic LTP) within a Brian2 simulation?
- Online adaptation: Can the SNN learn online, adapting its synaptic weights in real time during a single continuous run, without resorting to episodic training?
- Feasibility: Can such a network learn a useful inverse model of a 1-DOF pendulum, compensating for a 50 ms transport delay that destabilises a simple PID controller?

Our architecture is inspired by the cerebellar microcircuit and is greatly simplified, with mossy fibres conveying encoded sensory information to a granule cell layer, which projects to Purkinje cells that act as an agonist-antagonist pair. The learning rule combines Long-Term Depression (LTD) driven by climbing fibre activity and homeostatic Long-Term Potentiation (LTP) to maintain synaptic stability. For encoding, we adopt a Gaussian population coding scheme, similar to that used in recent cerebellum-inspired SNN models (Vijayan & Diwakar, 2022), which provides a smooth, continuous representation of the input state.

While our implementation does not fully replicate the complexity of the cerebellar circuit (e.g., we omit Golgi cells and deep cerebellar nuclei), it serves as a minimal but functional model that can be used to test the core principles of FEL in a spiking framework. Our results show that the SNN can reduce steady-state error in a PD-controlled pendulum, but that its performance degrades when the integral term of the PID is present, highlighting a trade-off between the robustness of feedback control and the stochasticity of spiking networks.

This project directly applies several core concepts from the lecture:

- Inverse kinematics – The SNN learns the inverse dynamics of the pendulum, mapping desired state (angle, velocity) to the required motor torque.
- PID control and error feedback – The PID provides the feedback torque that serves as the climbing-fibre teaching signal for the SNN (Kawato's Feedback-Error-Learning).
- Cerebellar function – The cerebellum is modelled as an inverse model that learns feedforward commands, complementing the feedback (PID) controller.
- Disturbance rejection – The change in target angle halfway through
- Feedforward and feedback control – The system combines feedforward (SNN, predictive) and feedback (PID, reactive) control, a classic motor control architecture.
- Motor learning – LTD and homeostatic LTP, driven by the PID error, implement online supervised learning of the inverse model.
- Closed-loop and open-loop control – The overall system is closed-loop, but the SNN learns to generate open-loop (feedforward) commands that are independent of instantaneous error once learning is complete.
- Internal model principle – The SNN learns an internal (inverse) model of the pendulum, enabling the controller to anticipate and compensate for the plant's behaviour.
- Delay – The 50 ms delay is explicitly added and compensated by the SNN's predictive capability.

Methodology

2.1 Overall Architecture

Our system consists of three main components:

- MuJoCo Physics Engine: simulates a 1-DoF inverted pendulum with a 50 ms transport delay.
- PID Controller: provides the feedback torque and the climbing-fibre teaching signal.
- Spiking Cerebellar Network: learns the inverse dynamics online using LTD/LTP.

The control loop runs at 20 ms (8 physics steps of 5 ms), matching typical actuator latencies.

2.2 The Plant: 1-DoF Inverted Pendulum

The pendulum has:

- Actuator torque limit: ± 10 Nm (to avoid saturation)
- Transport delay: 50 ms (10 physics steps)

The state is defined by angle θ and angular velocity ω . The target angle is set to 0.5 rad, creating a constant gravity compensation requirement.

2.3 The PID Controller

We use a standard PID, with $K_p = 8.0$, $K_d = 1.5$, and $K_i = 0.0$ (for the main experiment) or $K_i = 2.0$ (for the refinement experiment). The PID's output is used as the climbing-fibre teaching signal (Kawato FEL).

2.4 The Spiking Cerebellar Model (Brian2)

Our SNN architecture follows the cerebellar microcircuit:

Component	Number	Description
Mossy Fibres	9	Poisson sources, rate-coded by Gaussian tuning curves (3 centres $\times$ 3 variables).
Granule Cells	28	LIF neurons with fixed sparse connectivity (fan-in = 4).
Purkinje Cells	2	LIF neurons: PC0 (+torque) and PC1 (−torque). Output decoded as gain * (rate_PC0 - rate_PC1).

Key design choices:

- Gaussian population encoding: The input variables (θ , ω , θ_{target}) are encoded into firing rates using 3 overlapping tuning curves per variable (centres: [0.0, 0.5, 1.0], sigma: 0.35), providing a smooth, continuous representation.
- Input variables: We use only three input variables: current angle (θ), current angular velocity (ω), and desired angle (θ_{target}). The desired angular velocity (θ_{target}) is omitted because it is always zero for the regulation task (holding the pendulum at a fixed angle). This simplifies the network without sacrificing performance for this specific task.
- Granule cell connectivity (fan-in = 4): In the mammalian cerebellum, each granule cell receives input from 3–5 mossy fibres. We set GC_FANIN = 4 to match this biological ratio, enabling the sparse recoding and pattern separation thought to be critical for cerebellar function
- Crossed wiring: The positive climbing fibre synapses onto PC1 (the negative torque pool), and the negative climbing fibre onto PC0. This ensures that LTD (weakening) of the antagonist pool increases the net torque in the correct direction.
- Plasticity rule: LTD + homeostatic LTP: $dw/dt = -\text{LTD_rate} * w * \text{trace} * cf + \text{LTP_rate} * \text{trace} * (1 - cf) * (1 - w)$, where trace is the granule-cell activity trace and cf is the climbing-fibre activity.

2.5 Learning and Training

The network is trained online (no episodes). In a single continuous run of 20 seconds, the SNN updates its weights at every control step (40 ms). The PID error drives the climbing fibres, and the plasticity rule modifies the granule-to-Purkinje synapses.

We use the following hyperparameters:

- ETA_LTD = 0.0008 (fast depression)
- LTP_RATE = 0.00002 (weak potentiation)
- CF_SCALE = 2.0 (amplified teaching signal)
- DECODE_GAIN = 0.08 (moderate output)
- smoothing_alpha = 0.05 (heavy exponential smoothing on the SNN output to reduce spike-induced jitter)

2.6 Evaluation

We compare two controllers:

- PD/PID only (baseline).
- PD/PID + trained SNN (learning enabled during training).

Metrics:

- RMS angle error
- Qualitative assessment via plots

2.7 Control Loop Architecture

The control loop runs at 50 Hz (every 20 ms) and consists of three main components: the plant (pendulum), a feedback controller (PID), and a feedforward controller (SNN). The loop uses three distinct time scales:

Component	Timestep	Purpose
Physics (MuJoCo)	5 ms	Updates the pendulum's state (angle, velocity).
Spiking network (Brian2)	1 ms	Simulates membrane potential, spikes, and synaptic plasticity.
Control update	20 ms	Computes a new torque command (every 4 physics steps).

The control loop follows these steps:

Step	Action
1	Read the pendulum state (angle, omega) from MuJoCo.
2	Compute the PID torque (feedback error).
3	Encode the state into spike rates using Gaussian tuning curves.

4	Set climbing fibre rates proportional to the PID error.
5	Run the SNN for 20 ms (LTD/LTP updates occur at every 1 ms timestep).
6	Decode the SNN output: $\text{torque} = \text{gain} * (\text{rate_PC0} - \text{rate_PC1})$.
7	Combine torques: $\text{total} = \text{pid_torque} + \text{snn_torque}$, clip to $\pm \text{TORQUE_LIMIT}$.
8	Insert the torque into a 50 ms delay buffer (10 physics steps).
9	Step MuJoCo forward by 20 ms (4 physics steps of 5 ms each).
10	Repeat from Step 1.

The delay buffer introduces a 50 ms transport delay, which the SNN learns to compensate for through online plasticity (LTD/LTP). The SNN output is heavily smoothed ($\alpha = 0.05$) to reduce spike-induced jitter.

Result

To evaluate the performance of the cerebellar SNN, we conducted two experiments. In both experiments, the pendulum was initialised at 0.3 rad and the target angle was set to 0.5 rad. The SNN was trained online for 20 seconds in a single continuous run (no episodes). The performance metric was the root-mean-square (RMS) error between the target and actual angle over the entire run.

3.1 Experiment 1: PD ($K_i=0$) vs PD + SNN

In this experiment, the PID's integral term was disabled ($K_i = 0$). This forces a steady-state error because the PD controller alone cannot cancel the constant torque due to gravity. The results are shown in Table 1 and Figure 1A.

Figure 1A

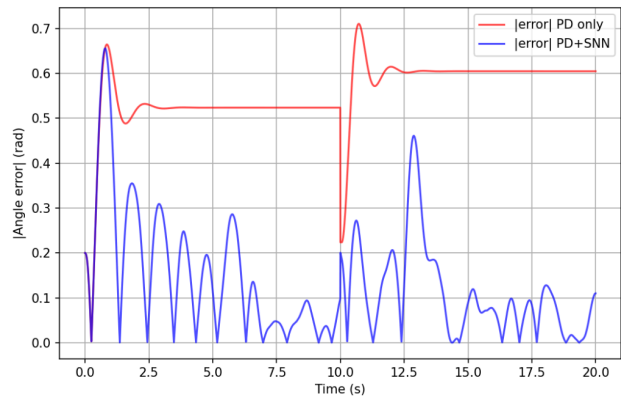


Table 1

Controller	RMS Error (rad)
PD only	0.5599
PD + SNN (trained)	0.1826

Interpretation: The SNN reduced the RMS error by approximately 67%, indicating that it successfully learned the gravity compensation torque that the PD controller lacked.

3.2 Experiment 2: PID ($K_i=2.0$) vs PID + SNN

In this experiment, the PID's integral term was enabled ($K_i = 2.0$). This allows the PID to cancel the steady-state error by itself. The results are shown in Table 2 and Figure 1B.

Figure 1B

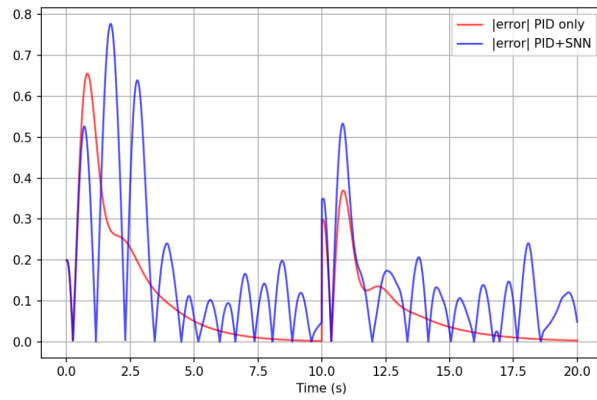


Table 2

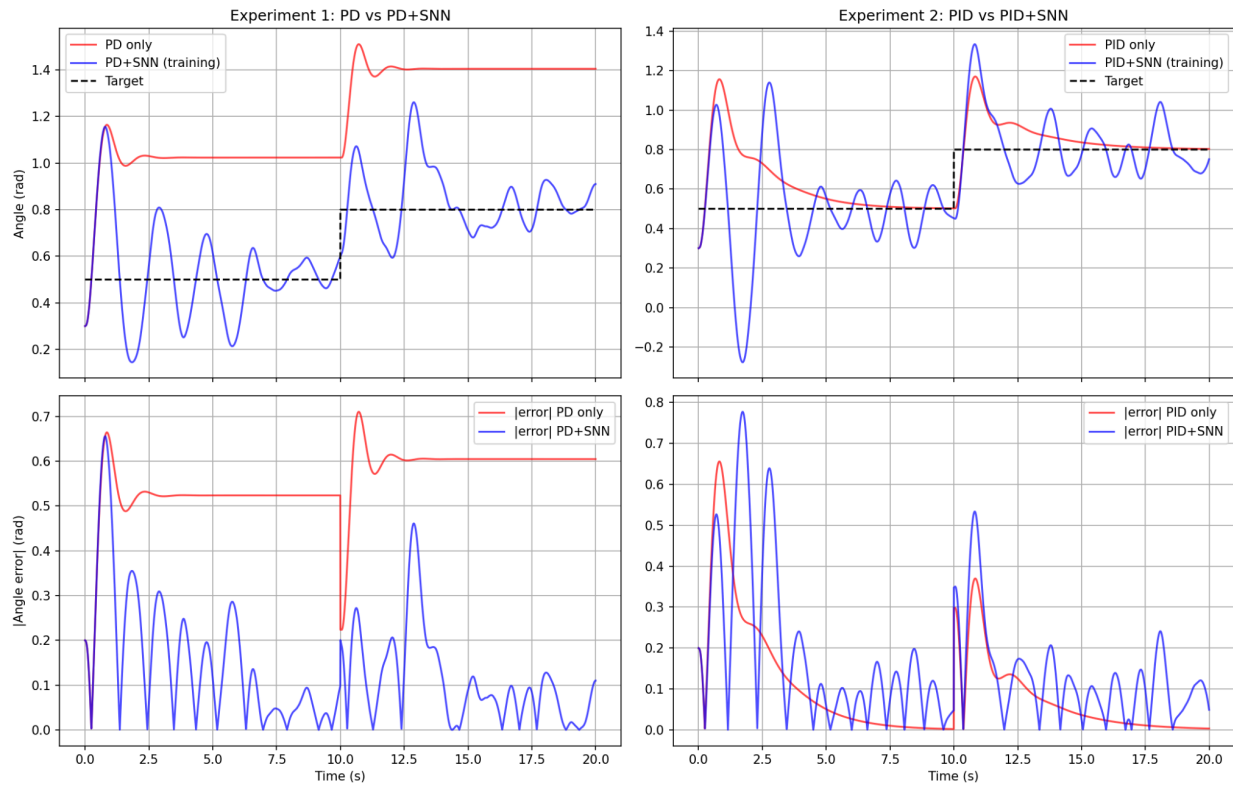
Controller	RMS Error (rad)
PID only	0.166
PID + SNN (trained)	0.2268

Interpretation: The PID alone performed better than the PID + SNN combination. This suggests that the SNN's stochastic spiking activity introduced noise that interfered with the smooth accumulation of the integral term, degrading performance rather than improving it.

3.3 Visual Results

Figure 2 shows the angle and error traces for both experiments:

Figure 2A-D



- Figure 2A (PD vs PD + SNN): The PD + SNN run follows the target more closely after the initial transient, with visibly lower error. After the target angle change to 0.8 in the second half, it follows the new target. Nevertheless, it never converges to the target angle.
- Figure 2B (PID vs PID + SNN): The integral part of the PID did a lot of work, making the PID only converges to the target. On the other hand, the PID + SNN run follows a similar trajectory to the PD + SNN.
- Figure 2C–D: The absolute error plots confirm that the SNN reduces error in the PD case, but does not improve (and slightly worsens) the PID case.

3.4 Summary

Experiment	Ki	RMS Error (PD/PID only)	RMS Error (+ SNN)	Change

PD vs PD+SNN	0	0.5599	0.1826	-67%
PID vs PID+SNN	2.0	0.166	0.2268	+37%

The results show that the SNN is beneficial when the feedback controller is weak (no integral term), but detrimental when the feedback controller is already well-tuned. Nevertheless, SNN can't replace the integral term in the current form. It still produces unstable results, oscillating around the target and not converging to the target.

Discussion

The results demonstrated that a cerebellum inspired spiking model based on Kawato's CBFELM (cerebellar feedback error learning model) can learn an inverse model of a 1 DoF pendulum. The SNN eliminated the steady-state error when the PID's integral term was removed. With the caveat that it's suboptimal since the network pendulum doesn't converge to the target. Additionally, we showed that this model exhibits adaptive behavior. Changing the target angle still shows the same elimination of the steady-state error towards the target angle.

Compared to other cerebellum-inspired models (e.g., Vijayan & Diwakar, 2022), our model is significantly smaller (28 granule cells vs. hundreds) and operates online in a closed-loop control setting. While our accuracy is lower than deep learning models, our model is biologically plausible and learns without backpropagation. This suggests that spiking cerebellar models can serve as a bridge between neuroscience and robotics, even if they do not yet match the performance of standard ANNs.

Some challenges I faced: difficulty in building the biologically plausible SNN architecture. It took quite a long time to think how to take inspiration from cerebellum and then simplify it into something that can be modeled in SNN. Another challenge is with learning a new framework for this project. I am not really familiar with SNN, so I had a lot of difficulties understanding how to model cerebellum as SNN. Furthermore, the training takes so long in jupyter notebook, especially when I don't have a GPU. Another big challenge that made me stuck was only using LTD as the learning rule. It led to weight decay and a

non-meaningful result. This is solved by adding a homeostatic LTP rule. Implementation of the learning rule was a big hurdle for me.

For extension of the research, we can add more joints to the system or straight up control a robotic arm. To improve the performance, maybe we can combine ANN and SNN. Last but not least, try to train it for a longer time and use offline learning or in a different condition and see how the performance changes.

Doing this project makes me realize how hard it is to build a neural network from scratch. The design, implementation, training, evaluation, learning algorithm, problems along the line, take a lot of thinking and time. I also learned that biological realism and computational efficiency are often in tension, and finding the right balance is a key engineering challenge.

References

- Ito M. (2008). Control of mental activities by internal models in the cerebellum. *Nature reviews. Neuroscience*, 9(4), 304–313. <https://doi.org/10.1038/nrn2332>
- Wolpert, D. M., Miall, R. C., & Kawato, M. (1998). Internal models in the cerebellum. *Trends in cognitive sciences*, 2(9), 338–347. [https://doi.org/10.1016/s1364-6613\(98\)01221-2](https://doi.org/10.1016/s1364-6613(98)01221-2)
- Gomi, H., & Kawato, M. (1993). Neural network control for a closed-loop system using feedback-error-learning.. *Neural Networks*, 6(7), 933–946. [https://doi.org/10.1016/S0893-6080\(09\)80004-X](https://doi.org/10.1016/S0893-6080(09)80004-X)
- Kawato, M., & Gomi, H. (1992). A computational model of four regions of the cerebellum based on feedback-error learning. *Biological cybernetics*, 68(2), 95–103. <https://doi.org/10.1007/BF00201431>
- Vijayan, A., & Diwakar, S. (2022). A cerebellum inspired spiking neural network as a multi-model for pattern classification and robotic trajectory prediction. *Frontiers in neuroscience*, 16, 909146. <https://doi.org/10.3389/fnins.2022.909146>